

FinLLM: Update for Kartik

Hi Kartik, here is where the project stands, what I built, and how to run it.

1. Where we landed on the idea

We kept the direction from your ChatGPT research: **a private, local AI assistant for fraud investigation and compliance (AML)**, as the proof of concept for the bigger “financial trust / risk” idea.

Design principle: **rules and ML detect, the LLM explains, a human decides**. Everything runs locally with Ollama, which is part of the pitch: compliance teams do not want customer data sent to outside AI APIs.

2. How it differs from the earlier prototype

I built this as a separate, structured project on my laptop. It uses the same core stack as the earlier prototype (Ollama, Qwen3, nomic-embed-text, ChromaDB), with these differences:

Earlier prototype	Now	Why
Whole PDF page as one embedding	900-character chunks with 150 overlap	Long pages were being truncated by the embedding model
Dense search only	Dense + LLM query rewrite + BM25 keyword search	Dense search missed exact regulatory phrases like “initial detection”
No testing	24-question evaluation with expected pages and wrong-answer checks	So every change is measured
qwen3:4b	qwen3:4b-instruct for most work	The default qwen3:4b tag is a thinking-only model: it always reasons first, which leaked into answers and was very slow
Default settings	temperature 0, context window 8,192, output cap 700 tokens	Repeatable answers; the default 4,096 context was silently cutting off the rules in our prompts

3. What exists now

Module 1: Regulation Q&A. Ask a question about the FFIEC Suspicious Activity Reporting guidance; get an answer with page citations, or a refusal if the document does not cover it. Every Q&A is logged.

Module 2: Transaction analysis.

- `generate_transactions.py`: synthetic data for 50 customers over 90 days, with 5 planted suspicious patterns (structuring, large foreign transfer, velocity burst, dormant account spike, spending spree).
- `detect.py`: rules with named reason codes plus an Isolation Forest anomaly model.
- `explain.py`: an alert narrative per flagged customer. Python prints the transactions and computes the “why unusual” section; the LLM writes only a short summary and open questions; code checks the output for invented transaction IDs or amounts and accusatory words.

See `README.md` for the full file list and `NOTES.md` for the detailed log of decisions.

4. Results

	Result
Module 1: correct page retrieved	20/20
Module 1: correct answers	19/20 with the thinking model (0 confident wrong); 17/20 with the instruct model (2 confident wrong)
Module 1: correct refusals on off-topic questions	4/4
Module 2 detector on 5 unseen datasets	precision 92% +/- 5%, recall 87% +/- 8%, 59/60 suspicious customers caught
Module 2 narratives	16/16 complete, 15/16 passed checks, about 6 seconds each

Caveat: one document and synthetic data. These show the approach works, not real-world performance.

5. How to run it on your Windows laptop

```
git clone https://github.com/Prashasti9/FinLLM.git
cd FinLLM
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

ollama pull qwen3:4b-instruct
ollama pull nomic-embed-text
ollama pull qwen3:4b

python check_setup.py
```

Then:

```
python ingest.py          # build the document index (once)
python ask.py             # ask questions; type exit to quit
python eval.py --quick    # 5-question check (a few minutes)

python generate_transactions.py # synthetic data
python detect.py           # flags
python eval_detect.py      # detector scores
python explain.py          # narratives for the top 3 customers
```

Note: the full `eval.py` takes a long time on a laptop (about a minute per question). Use `--quick` for everyday checks.

6. Open problems

- The faster instruct model gave 2 confidently wrong answers that the thinking model got right. Plan: use the thinking model for regulation Q&A and the instruct model for narratives.
- One question (whether a bank can tell a customer a SAR was filed) is refused because the right page is not retrieved. A safe failure, but a gap.
- Local inference is slow on a laptop.
- No supervised model yet, and only one regulatory document.

7. Next steps

- Split models by task (accurate model for regulation Q&A, fast model for narratives) and reduce latency, measured with `eval.py --quick`.
- Module 3: link each flag to the relevant regulation page, with citations.
- Train a supervised model on a public labeled dataset (PaySim or IBM AML) and compare it with the rules and Isolation Forest.
- Add more regulatory documents and extend the eval questions.
- Talk to 3-5 people who work AML alerts: what takes their time, and what would they trust an AI to do.

8. How we should work together

-
- You are already added as a collaborator on GitHub.
 - Pull before starting: `git pull`. Work on a branch for bigger changes: `git checkout -b your-feature`.
 - Run the relevant eval before and after any change, and note the numbers in `NOTES.md`.
 - Keep `requirements.txt` updated when adding a package.
 - Please add a line to the README “Authors” section describing what you built.